BHDD: A Burmese Handwritten Digit Dataset

Swan Htet Aung*, Hein Htet, Htoo Say Wah Khaing, Thuya Myo Nyunt
Expa.AI Research Team
Yangon, Myanmar
*Corresponding author: swan@expa.ai

***Abstract*—We introduce the Burmese Handwritten Digit Dataset (BHDD), a collection of 87,561 grayscale images of handwritten Burmese digits in ten classes. Each image is 28×28 pixels, following the MNIST format. The training set has 60,000 samples split evenly across classes; the test set has 27,561 samples with class frequencies as they arose during collection. Over 150 people of different ages and backgrounds contributed samples. We analyze the dataset's class distribution, pixel statistics, and morphological variation, and identify digit pairs that are easily confused due to the round shapes of the Myanmar script. Simple baselines (an MLP, a two-layer CNN, and an improved CNN with batch normalization and augmentation) reach 99.40%, 99.75%, and 99.83% test accuracy respectively. BHDD is available under LGPL-3.0 at https://github.com/baseresearch/BHDD.**

***Index Terms*—handwritten digit recognition, benchmark dataset, Myanmar script, Burmese digits, optical character recognition**

I. Introduction

MNIST [1] set the standard for handwritten digit recognition in 1998: 70,000 images of Latin digits, each 28×28 grayscale pixels, with a fixed train/test split. It has been used in thousands of studies since. EMNIST [2] extended the same format to Latin letters, and Kuzushiji-MNIST [3] showed that Japanese cursive characters produce very different error patterns than Latin digits; different scripts need their own benchmarks.

Myanmar has roughly 55 million people. Burmese, the main language, is spoken natively by over 33 million. The Myanmar script, called *sar-lone* (literally "round script"), developed its circular letterforms because it was traditionally written on palm leaves, where straight strokes would tear through the leaf. The digits carry over this roundness: they are built from curves, loops, and arcs, and several pairs look similar enough to confuse classifiers.

Until now, no public benchmark dataset existed for Burmese handwritten digits. We release BHDD to fill that gap. The dataset contains 87,561 labeled images from over 150 contributors. We also provide a detailed analysis of the data and three baseline models.

II. Related Work

MNIST [1] has 70,000 handwritten Latin digit images and remains the most common benchmark for digit recognition. Its fixed format (28×28 grayscale, integer labels, standard split) made it easy to compare results across models. EMNIST [2] added handwritten letters in the same format.

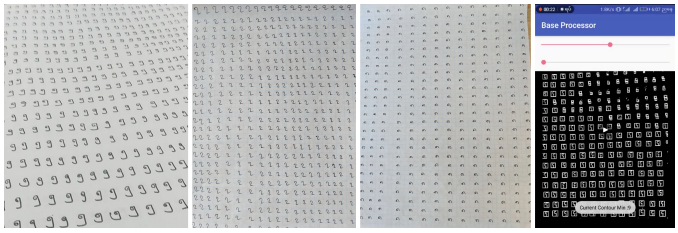

Fig. 1. Sample collection sheets on plain paper (left) and grid paper (middle two), and the mobile preprocessing app (right) that let contributors verify digit extraction before submission.

Kuzushiji-MNIST [3] took the same approach for cursive Japanese characters and found that the error patterns looked nothing like those on Latin digits. This made the broader point that writing systems with different visual structures need their own datasets. Analogous benchmarks have since been developed for other scripts, including Bengali [4] and Arabic [5] handwritten digits.

No public benchmark for Myanmar handwritten digits existed before BHDD.

III. Dataset

A. Collection Methodology

We organized a community collection effort through the Expa.AI Research Team. Over 150 people wrote Burmese digits on plain A4 paper (both white and yellow sheets), each filling approximately ten pages (one per digit class). Contributors came from Taungoo Computer University (volunteers and interns), St. Augustine / B.E.H.S (2) Kamayut (high school students), and friends and family of the research team. Ages ranged from teenagers to people in their 50s, with occupations including clerks, programmers, and students. This range of contributors produces natural variation in stroke thickness, pen pressure, and letter formation across the dataset. Fig. 1 shows representative collection sheets.

The filled sheets were scanned or photographed. Because many contributors had low-quality phone cameras, we developed a mobile application that applied adaptive thresholding and contour detection in real time, letting users adjust parameters and retake photos until the extracted digits were clean. The preprocessing pipeline converted images to grayscale, binarized with adaptive thresholding, found contours to locate each digit, cropped to the bounding box, centered in a 28×28 frame, and normalized intensity. Quality assurance was a

two-stage process: all 20 members of the annotation team independently inspected every training page, and two data engineers then performed a final verification pass over the complete dataset. Mislabeled, illegible, and duplicate images were removed at each stage. The training and test sets use different contributors so that models cannot rely on memorizing individual handwriting styles. Most contributors were based in Yangon, with smaller groups from Mandalay, Nay Pyi Taw, Shan State, and the United States.

B. Data Format and Structure

Each sample is a 28×28 grayscale image (uint8, values 0–255) with an integer label from 0 to 9. The dataset is available in Python pickle and gzip-compressed IDX formats; the IDX files work directly with any MNIST data loader. Fig. 2 shows samples from all ten classes.

C. Class Distribution

The dataset contains 87,561 images: 60,000 for training and 27,561 for testing. The split is by contributor; all pages from a given person go entirely to one set, so no writer's handwriting appears in both. The training set is downsampled to exactly 6,000 samples per class. The test set is left unbalanced (Fig. 3), with per-class counts ranging from 6,856 (class 0) to 389 (class 9), reflecting how often each digit appeared in the collected sheets.

D. Visual and Statistical Analysis

Pixel intensity. Mean pixel intensity per class ranges from 12.5 (class 2) to 26.5 (class 0). Ink coverage (the fraction of non-zero pixels) ranges from 30.4% (class 2, a thin hook) to 56.8% (class 0, a full circle). Per-class distributions are shown in Fig. 4.

Mean images. Averaging all 6,000 training samples per class gives the images in Fig. 5. Each class has a distinct shape: class 0 is a ring, class 1 is an open arc, class 8 is a spiral. The averages are sharp rather than blurry, meaning writers agree on the basic form of each digit.

Variance. Fig. 6 maps where pixel values vary most across writers. High-variance spots sit at stroke endpoints and junctions, where individual handwriting styles diverge most. Complex digits like class 8 show wide variance regions; simple ones like class 2 show a narrow band.

Writing styles. Fig. 7 shows individual samples from four classes with high within-class variation (0, 3, 5, 8). Stroke thickness, curvature, and how far the loop closes all differ from writer to writer.

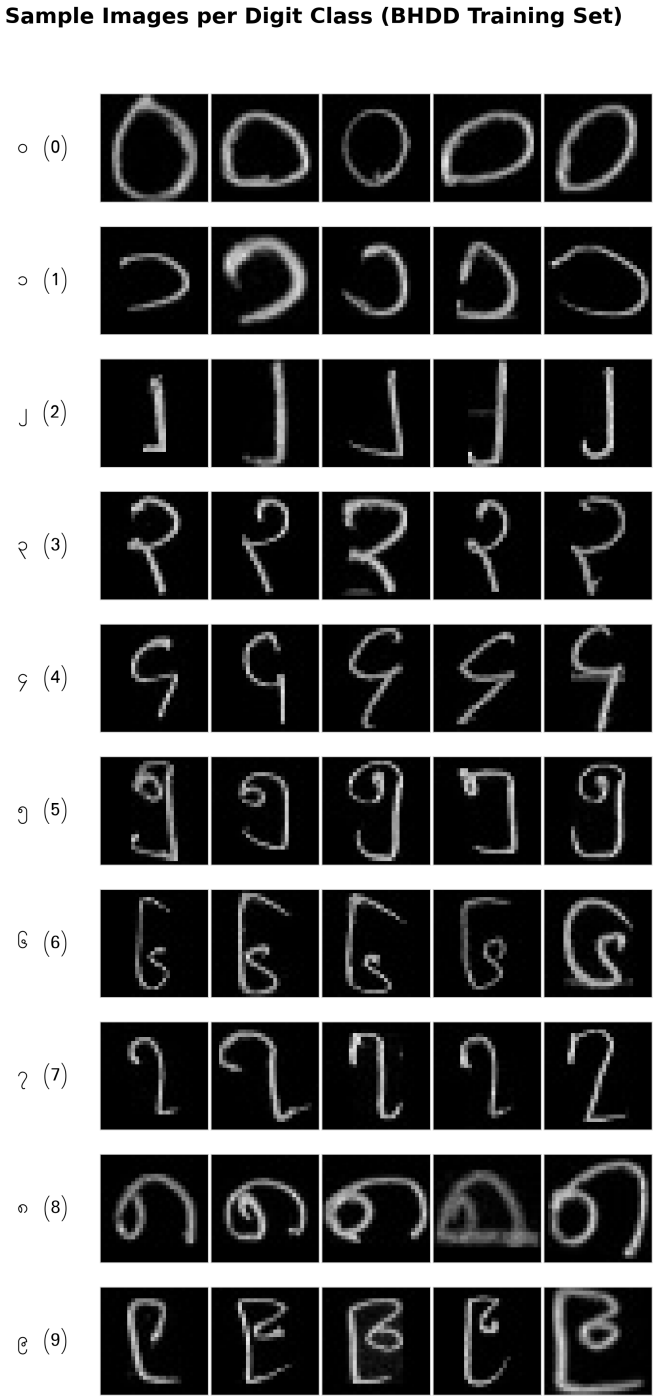

Fig. 2. Samples from all ten Burmese digit classes (rows: 0–9).

E. Script-Specific Challenges

Because Myanmar digits are built from round strokes, some pairs look very similar. We identified the worst pairs from our improved CNN's confusion matrix (Section IV-C) and show them in Fig. 8.

Digits 0 and 1 are the hardest pair: 24 misclassifications in both directions. The only difference is whether the circle is closed or has a small gap. Digits 0 and 8 share a round outer shape but differ inside (plain ring vs. spiral), with 8 errors both ways. Digit 3 gets mistaken for 1 seven times when its tail is faint and only the curved head shows. Digits 5 and 8 sometimes overlap because both have looped forms (3 errors).

When digits share curved sub-strokes, small differences in pen pressure or whether someone fully closes a loop can shift a sample's appearance toward a neighboring class.

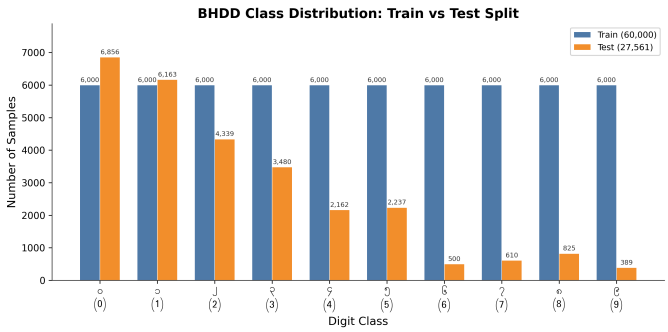

Fig. 3. Training set (balanced, 6,000 per class) vs. test set (unbalanced, 6,856 down to 389).

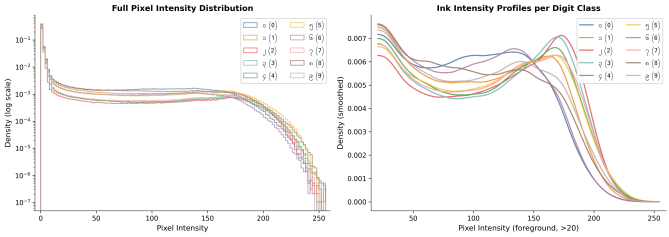

Fig. 4. Pixel intensity distributions per class. Left: log-scale histogram. Right: per-class kernel density estimates.

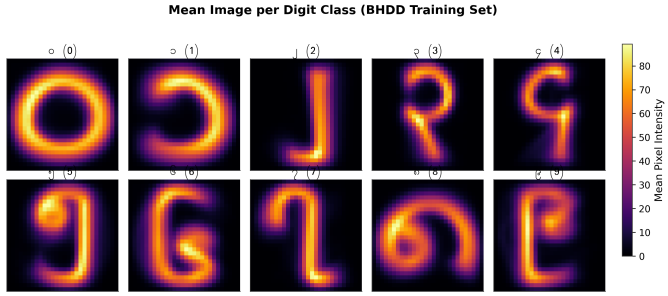

Fig. 5. Mean image per class (6,000 training samples each). Sharp shapes show consistent digit forms across writers.

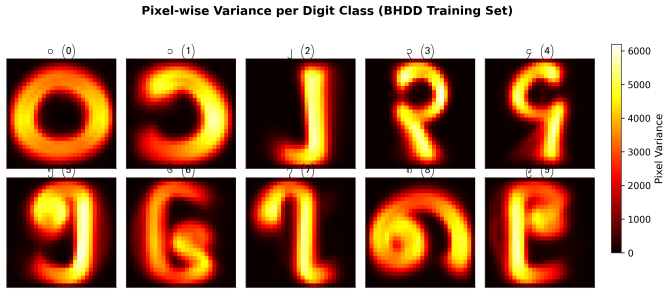

Fig. 6. Per-class pixel variance. Bright areas mark where writers differ most.

IV. Baseline Experiments

We trained three models to establish initial performance on the dataset. All runs used seed 42.

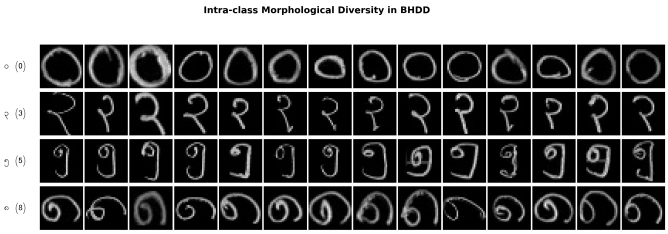

Fig. 7. Writing style variation within classes 0, 3, 5, and 8.

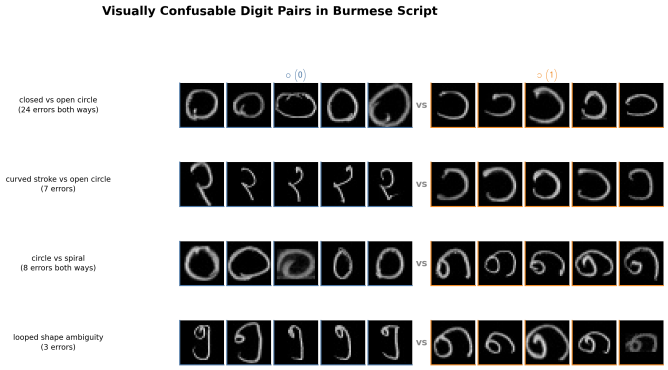

Fig. 8. Commonly confused digit pairs with error counts from the improved CNN.

A. Models

MLP. Two hidden layers (256, 128 units), ReLU, Adam, trained on flattened 784-d input with early stopping (10% validation split, up to 50 epochs). Implemented in scikit-learn [6].

CNN. Two conv layers (32 and 64 filters, 3×3, ReLU, 2×2 max-pool), dropout (0.25 spatial, 0.5 dense), one FC layer (128 units), 10-way output. Trained 15 epochs with Adam ($\text{lr} = 10^{-3}$) and cross-entropy. Implemented in PyTorch [7].

Improved CNN. Three conv layers (32, 32, 64 filters) with batch normalization after each, two max-pool stages, and a 128-unit FC layer (431K parameters, barely more than the baseline's 421K). Trained 25 epochs with on-the-fly augmentation (rotation $\pm15°$, translation ±2 px, scale 0.9–1.1$\times$), cosine learning-rate annealing, and light weight decay.

B. Results

All three models exceed 99% accuracy with standard training procedures (Table I). The improved CNN reaches 99.83% with a macro F1 of 0.998, cutting misclassifications roughly in half compared to the baseline CNN while adding only 10K parameters.

C. Error Analysis

The improved CNN's confusion matrix (Fig. 9) has nearly all off-diagonal entries at zero. Of 27,561 test samples, only 47 are misclassified. The errors cluster around digits 0 and 1 (a closed versus open circle), which account for about a third of all mistakes, consistent with the structural similarity between these two classes.

TABLE I
BASELINE RESULTS ON THE BHDD TEST SET

Model	Accuracy	F1	Precision	Recall
MLP	0.9940	0.9905	0.9876	0.9934
CNN	0.9975	0.9964	0.9959	0.9970
Improved CNN	0.9983	0.9980	0.9972	0.9988

F1, Precision, and Recall are macro-averaged.

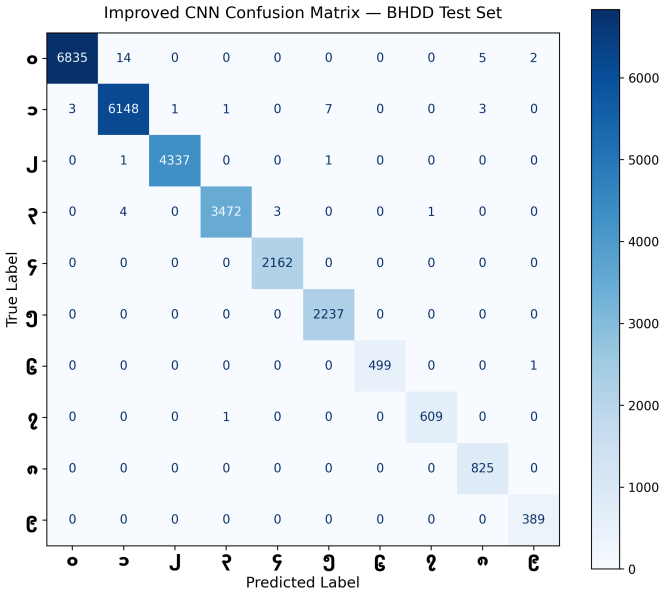

Fig. 9. Improved CNN confusion matrix. Nearly all off-diagonal entries are zero. The 0–1 pair dominates the remaining errors.

V. AVAILABILITY

BHDD is at https://github.com/baseresearch/BHDD under LGPL-3.0, in pickle and IDX formats. The repository includes exploration scripts, baseline code, and usage examples.

VI. CONCLUSION AND FUTURE WORK

We have released BHDD, a dataset of 87,561 Burmese handwritten digit images from over 150 contributors. The training set is balanced; the test set preserves the class frequencies as they arose during collection. Simple baselines reach up to 99.83% accuracy, and the remaining errors arise from visual similarity between round digits that share curved sub-strokes.

Next, we plan to expand the dataset to cover Myanmar consonants and characters, collect word- and sentence-level handwriting, and study how test-set imbalance affects evaluation.

ACKNOWLEDGMENTS

We thank the volunteers from Taungoo Computer University, the students of St. Augustine / B.E.H.S (2) Kamayut, and everyone who contributed handwriting samples.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *International Joint Conference on Neural Networks (IJCNN)*, 2017, pp. 2921–2926.

[3] T. Clanuwat, M. Bober-Irizar, A. Kitamoto, A. Lamb, K. Yamamoto, and D. Ha, "Deep learning for classical Japanese literature," *arXiv preprint arXiv:1812.01718*, 2018.

[4] S. Alam, T. Reasat, R. M. Doha, and A. I. Humayun, "NumtaDB – assembled Bengali handwritten digits," *arXiv preprint arXiv:1806.02452*, 2018.

[5] A. El-Sawy, M. Loey, and H. El-Bakry, "Arabic handwritten characters recognition using convolutional neural network," *WSEAS Transactions on Computer Research*, vol. 5, pp. 11–19, 2017.

[6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.